

Hypo-Stage: Tool-Supported Architectural Hypothesis Engineering in Professional Software Development Teams — A Case Study

José Gonçalves Lima Neto

11 June 2026

Master's Thesis Defense

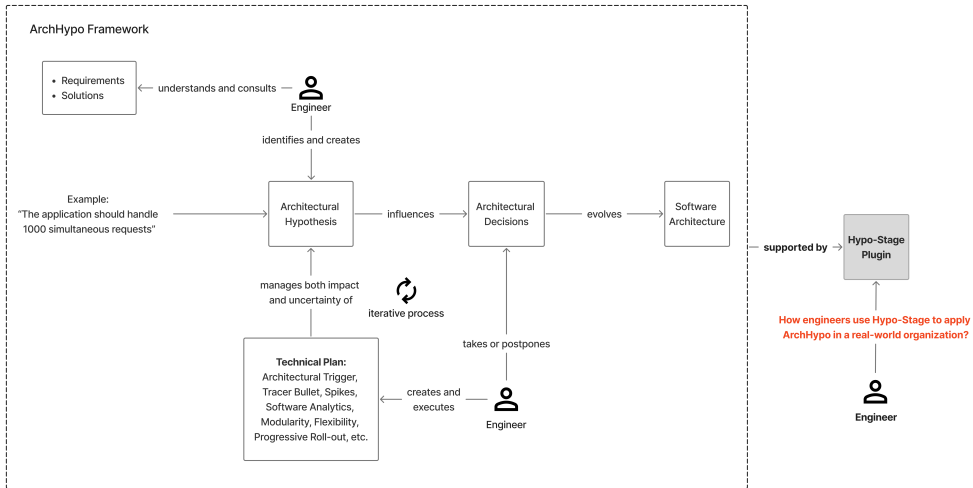
Advisor: Prof. Dr. Paulo Roberto Miranda Meirelles

Graduate Program in Computer Science

IME – University of São Paulo



Context



Problem & research questions

The gap

Architectural uncertainty is unavoidable, yet it stays **implicit** and locked in a few experts.

ArchHypo makes it explicit — but teams find it **hard to learn**.

Hypo-Stage already provides tool support, yet there is **no real-world usage** to observe its practice.

RQ1. How do teams *use* Hypo-Stage and ArchHypo on a scaled product?

RQ2. What *uncertainties* and recurring *patterns* do they record?

RQ3. What *benefits and challenges* do team members perceive?

RQ4. Which *refinements* to the tool emerge, and how do they address the challenges?

Method & study setup

Single organization · scaled product · Backstage IDP

Product team

12 engineers

Platform team

4 engineers

Five 2-week sprints (≈9 weeks)

Sprint 1

Sprint 2

Sprint 3

Sprint 4

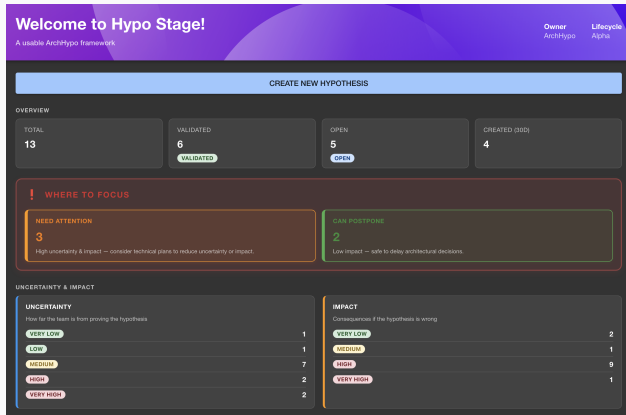
Sprint 5

Participant
observation

Artifact analysis
(13 hypotheses)

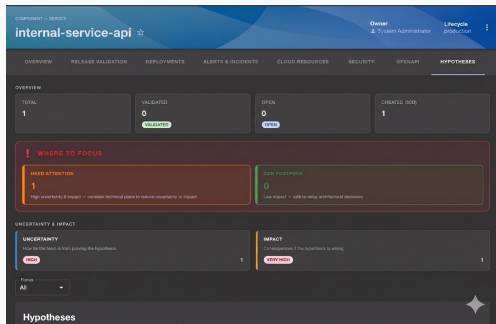
Active-listening
sessions

Three triangulated data sources



Adoption was **senior-led**
and **event-anchored**:

- 12 of 13 hypotheses authored by senior/staff engineers
- Registration clustered around protected technical sessions
- Backstage integration cut context-switching cost

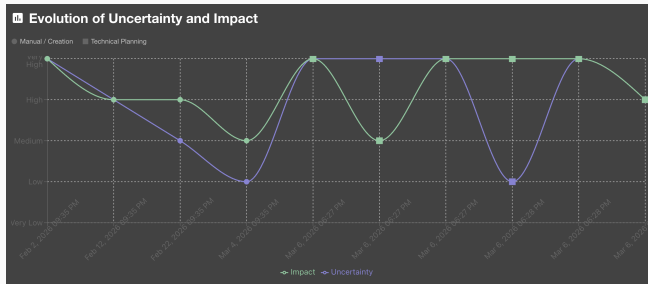


Uncertainty $\times$ impact, surfaced per service in the developer portal. Dominant quality attributes: **Reliability** and **Auditability**.

Five emerging hypothesis patterns

- **Hypothesis Cluster** — one platform concern $\rightarrow$ linked hypotheses
- **Regulatory Trigger** — compliance obligation as tracked uncertainty
- **Context-Agnostic Restructuring** — whether/how to decouple boundaries
- **Load-Validated** — proven under load, implementation still open
- **Shared Resource Governance** — ownership rules for shared infrastructure

First empirically grounded answer to *what shapes* architectural hypotheses take in practice.



Evolution chart: uncertainty & impact tracked across successive assessments.

+ Create New Hypothesis

Define your hypothesis and assess its uncertainty and potential impact

Entity References

Please select at least one entity reference. Type to search existing components.

Hypothesis Statement*

8/500 characters

Source Type*

Uncertainty Level

How far is the team from validating/falsifying this hypothesis?

1 2 3 4 5 Not rated

Impact Level

What is the potential impact on the system if the hypothesis proves false?

1 2 3 4 5 Not rated

Quality Attributes

Quality Attributes (select one or more)

Please select at least one quality attribute that this hypothesis affects. Click the field to open the list; each option shows a short concept in parentheses.

Inline formulation guidance in the creation form lowers the falsifiability barrier.

9 refinement categories — including prioritization, the evolution chart, and form usability — each traced to an observed friction point.

Implications & lessons learned

Finding

Even with tool support, hypothesis work stayed **senior-anchored** (12 of 13).

Implication

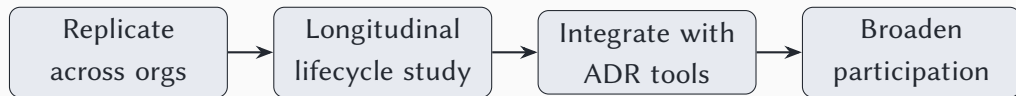
A tool removes **friction** but cannot supply the **organizational conditions** for broad participation.

Lesson

Ship hypothesis support **inside** the developer portal — *extend* the workflow, don't add a new one.

Distributed architectural work needs lightweight structure that preserves team autonomy while protecting shared decision quality.

Future work & closing



Takeaway

Tool-mediated ArchHypo succeeds at **integration and traceability**; the remaining challenge is **organizational and pedagogical**.

Limitations: single organization, 13 hypotheses, ≈ 9 weeks, researcher as participant — emerging propositions, not consolidated findings.